

```
#Dateien
ChristmasSales = pd.read_csv('Christmas Sales and Trends.csv')
ChristmasSales = pd.read_excel('Christmas Sales and Trends.xlsx')
#Ansich Datensatz erste und letzte Zeilen
display(ChristmasSales.head(3))
display(ChristmasSales.tail(3))
#Mit .describe können die metrischen Merkmale ausgegeben werden
ChristmasSales.describe()
# allgemeine Infos zum Datensatz
display(datensatz.info())

# --- DATENBEREINIGUNG
# Zeilen mit fehlenden Werten löschen
df = df.drop()
df = df.drop_duplicates(keep='first')

# --- STICHPROBENUMFANG (Wie viele Messungen brauche ich?) ---
# z=1.96 (für 95% Sicherheit), sigma=bekannte Streuung, e=erlaubte Abweichung
z = 1.96
e = 0.05
n_notwendig = ((z * sigma) / e)**2
print(f"Benötigtes n: {np.ceil(n_notwendig)}")

# --- SKALIERUNG / STANDARDISIERUNG (Penguin Übung) ---
# Daten auf Mittelwert 0 und Std 1 bringen (Z-Transformation)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[['length', 'weight']])
```

```
# ECDF PLOT (Empirische Verteilungsfunktion)
# Zeigt den kumulierten Anteil der Datenpunkte bis zu einem Wert.
# Die Kurve startet bei 0 und endet bei 1. Bei 0.5 auf der y-Achse liegt der Median.
# Einstellungen: x=Spalte, hue=Vergleich von Gruppen
sn.ecdfplot(data=df, x='length', hue='group')
plt.show()

# BOXPLOT (Gruppenvergleich)
# Visualisiert Median, Quartile und Ausreißer. Boxen zeigen die Streuung.
# Überlappen sich Boxen nicht, liegen signifikante Unterschiede nahe.
# - Mittellinie: Median
# - Box-Kanten: Q1 (25%) und Q3 (75%). Die Box enthält 50% der Daten.
# - Whiskers: Gehen bis zum letzten Datenpunkt innerhalb von 1.5 * IQR.
# - Punkte: Ausreißer außerhalb der Whisker-Grenzen.
# Einstellungen: x=Kategorie, y=Messwert, hue=Farbe pro Gruppe
sn.boxplot(data=df, x='group', y='length', hue='group')
plt.show()

# HISTOGRAMM (Dichteverteilung)
# Stellt Häufigkeiten in Klassen dar. Prüft die Normalverteilung.
# Einstellungen: bins=Anzahl der Balken, kde=True (zeichnet Glättungslinie)
sn.histplot(df['length'], kde=True, bins=30)
plt.show()

# COUNTPLOT (Absolute Häufigkeit)
# Erstellt Balkendiagramme für kategoriale Daten (wie Gruppen oder Ja/Nein).
# Zeigt direkt an, welche Kategorie am häufigsten vorkommt.
# Einstellungen: x=Spalte
sn.countplot(data=df, x='group')
plt.show()

# BARPLOT (Relative Häufigkeit/Mittelwerte)
# Zeigt Anteile oder Durchschnittswerte pro Gruppe.
# Einstellungen: x=Gruppe, y=Wert, estimator=np.mean oder len
sn.barpplot(data=df, x='group', y='length', estimator=len)
plt.show()

# REGPLOT (Streudiagramm mit Trend)
# Zeigt den linearen Zusammenhang zwischen zwei numerischen Werten.
# Punkte nah an der Linie bedeuten eine hohe Korrelation.
# Einstellungen: x=Variable 1, y=Variable 2
sn.regplot(data=df, x='Accept', y='Apps')
plt.show()

# HEATMAP (Korrelationsübersicht)
# Farbmatrix der Zusammenhänge. Werte nahe 1 zeigen starke Bindung.
# Erfordert numerische Daten. Filtert Spalten vorher aus.
# Einstellungen: annot=True (schreibt Zahlen in Felder), cmap=Farbschema
sn.heatmap(df.corr(numeric_only=True), annot=True, cmap='RdBu')
plt.show()

# PAIRPLOT (Multivariate Analyse)
# Erstellt eine Matrix aller Variablenkombinationen.
# Hilft beim Finden von Mustern in großen Datensätzen.
# Einstellungen: kind='reg' (fügt Regressionslinien in jedes Feld ein)
sn.pairplot(df.select_dtypes('number'), kind='reg')
plt.show()
```

```
REGRESSION & MODELLQUATE
P-Wert (P>|t|): < 0.05 -> Merkmal ist signifikant (wichtig).

R-squared: Anteil erklärter Varianz (0 bis 1). Je höher, desto besser.

F-Statistik: Modell insgesamt brauchbar (wenn Prob < 0.05).
F-Statistik prüft ebenfalls Nullhypothese (dass alle Koeffizienten gleich Null sind)

R2-adj (0.91): 91% Varianz kann durch die unabhängigen Merkmale im Modell erklärt werden
Die Vorhersagen liegen nah an den tatsächlichen Werten.
Adjustiert: Korrigiert um Variablenanzahl (verhindert Schoenrechnen).

# --- SATZ VON BAYES (Beispiel: Restaurant Ratte) ---
# Berechnet die Wahrscheinlichkeit für eine Ratte, wenn eine schlechte Bewertung vorliegt.

# Gegebene Wahrscheinlichkeiten
p_ratte = 0.10 # 10% der Restaurants haben Ratten (P(A))
p_schlecht_bei_ratte = 0.80 # 80% bekommen schlechte Bewertung, WENN Ratte da ist (P(B|A))
p_schlecht_ohne_ratte = 0.20 # 20% bekommen schlechte Bewertung, WENN KEINE Ratte da ist (P(B|nicht A))

# 1. Totale Wahrscheinlichkeit für eine schlechte Bewertung berechnen (P(B))
p_schlecht_total = (p_schlecht_bei_ratte * p_ratte) + (p_schlecht_ohne_ratte * (1 - p_ratte))

# 2. Satz von Bayes anwenden (P(A|B))
p_ratte_bei_schlecht = (p_schlecht_bei_ratte * p_ratte) / p_schlecht_total
print(f"Wahrscheinlichkeit für Ratte bei schlechter Bewertung: {p_ratte_bei_schlecht:.4f}")

# --- FEHLER 1. ART UND 2. ART (Alpha & Beta) ---
# Fehler 1. Art (Alpha): Du sagst "Prozess ist kaputt" (H1), obwohl alles passt (H0).
# Fehler 2. Art (Beta): Du sagst "Alles passt" (H0), obwohl der Prozess kaputt ist (H1).

# Führe einen t-Test durch (Sollwertprüfung)
test = pg.ttest(df['length'], 115)

# Werte aus dem Test extrahieren
p_val = test['p-val'].values[0]
power = test['power'].values[0] # Teststärke entspricht 1 - Beta

alpha = 0.05 # Festgelegtes Signifikanzniveau
beta = 1 - power # Wahrscheinlichkeit für Fehler 2. Art

# Logik zur Identifikation des Fehlerrisikos
if p_val < alpha:
    print("H0 wird abgelehnt. Signifikanter Unterschied.")
    print(f"Mögliches Risiko: Fehler 1. Art. Wahrscheinlichkeit maximal: {alpha}")
else:
    print("H0 wird beibehalten. Kein signifikanter Unterschied.")
    print(f"Mögliches Risiko: Fehler 2. Art. Wahrscheinlichkeit: {beta:.4f}")
```

```
SKALENNIVEAUS UND DATENTYPEN
Daten haben unterschiedliche Niveaus. Das Niveau bestimmt die möglichen Berechnungen.

1. Nominalskala. Diese Daten sind Namen oder Kategorien ohne logische Reihenfolge. Ein Durchschnitt ergibt keinen Sinn. Beispiele sind Farben, Geschlecht oder Postleitzahlen.

2. Ordinalskala. Diese Daten haben eine logische Rangfolge. Die Abstände zwischen den Stufen sind undefiniert. Ein Beispiel sind Schulnoten. Eine Note 1 ist besser als eine 2. Der Abstand zwischen 1 und 2 ist nicht zwingend gleich wie zwischen 3 und 4.

3. Metrische Skala. Diese Daten sind Zahlenwerte mit definierten Abständen. Du berechnest Durchschnitt und Streuung. Beispiele sind Länge, Gewicht oder Preis.
```

```
VERTEILUNGSFORMEN
Daten verteilen sich in bestimmten Mustern. Du misst die exakte Form dieser Verteilung.

1. Schiefe. Sie beschreibt die Asymmetrie der Daten. Ein Wert von 0 bedeutet eine exakt symmetrische Verteilung. Ein negativer Wert bedeutet eine linksschiefe Verteilung. Die Masse der Daten liegt auf der rechten Seite. Ein positiver Wert bedeutet eine rechtsschiefe Verteilung. Die Masse der Daten liegt auf der linken Seite.

2. Wölbung. Sie beschreibt die Spitzigkeit der Verteilung im Vergleich zur Normalverteilung. Ein positiver Wert bedeutet eine extrem spitze Kurve mit dicken Rändern. Die Werte konzentrieren sich stark um den Mittelwert.
```

```
Der zentrale Grenzwertsatz besagt, dass die Summe oder der Durchschnitt einer großen Anzahl von unabhängigen Zufallsvariablen annähernd normalverteilt ist. Dies gilt unabhängig davon, welche Verteilung die ursprünglichen Variablen hatten. Dabei muss der Datensatz eine große genug große haben (n>30)

SCHLIESSENDE STATISTIK
Du nutzt eine kleine Stichprobe für sichere Aussagen über die große Grundgesamtheit.

1. Punktschätzer. Du schätzt einen unbekannten Wert der Realität mit einem einzigen berechneten Wert aus deinen Daten. Der Stichprobenmittler schätzt den echten Erwartungswert der Grundgesamtheit.

2. Konfidenzintervall. Ein Punktschätzer ist selten exakt. Das Konfidenzintervall liefert einen Bereich. Der echte Wert liegt mit einer definierten Sicherheit in diesem Bereich. Bei einem 95 Prozent Intervall bist du dir zu 95 Prozent sicher.

3. Hypothesentest. Du triffst eine mathematische Annahme über die Realität. Diese Annahme ist die Nullhypothese. Die Gegenthese ist die Alternativhypothese. Der Test prüft deine Daten strikt gegen die Nullhypothese.

4. Fehler erster Art. Du lehnt die Nullhypothese ab. In Wahrheit stimmt sie. Du schlägst falschen Alarm. Die Wahrscheinlichkeit dafür definierst du als Signifikanzniveau Alpha.

5. Fehler zweiter Art. Du behältst die Nullhypothese bei. In Wahrheit ist sie falsch. Du übersiehst einen echten Effekt. Die Wahrscheinlichkeit dafür nennt sich Beta.

6. p Wert. Dieser Wert ist das direkte Ergebnis deines Tests. Er zeigt die Wahrscheinlichkeit für deine Daten unter der Annahme der Nullhypothese. Ist der p Wert kleiner als dein Signifikanzniveau Alpha, verwirfst du die Nullhypothese. Du erzielst ein signifikantes Ergebnis.
```

```
REGRESSION (STATSMODELS SUMMARY)
# coef (const): Der y-Achsenabschnitt (Intercept). Das ist der theoretische Wert der Zielvariablen, wenn alle anderen Merkmale exakt 0 sind.
# coef (Merkmale): Die Steigung. Steigt dieses Merkmal um 1 Einheit, ändert sich die Zielvariable exakt um diesen Wert.
# std err (Standardfehler): Zeigt die Unsicherheit der Koeffizienten-Schätzung. Ein kleinerer Wert bedeutet eine präzisere Schätzung.
# t-Wert (t): Berechnet sich aus coef / std err. Je weiter dieser Wert von 0 entfernt ist, desto sicherer hat das Merkmal einen echten Einfluss.
# [0.025 0.975] (Konfidenzintervall des Koeffizienten): Wenn die Zahl 0 nicht in diesem Intervall liegt, ist das Merkmal statistisch signifikant für das Modell.
```

```
# Einzelne Spalte extrahieren
total_price = ChristmasSales['TotalPrice']
# Mehrere Spalten auswählen (beachte die doppelten Klammern [[ ]])
df_klein = ChristmasSales[['TotalPrice', 'Quantity', 'UnitPrice']]
# Daten Filtern
df_food = ChristmasSales[ChristmasSales['Category'] == 'Food']
df_teuer = ChristmasSales[ChristmasSales['TotalPrice'] > 100]
df_mix = ChristmasSales[(ChristmasSales['Category'] == 'Food') & (ChristmasSales['Quantity'] > 5)]

#Zusammenhang kategorialer Merkmale
# Kreuztabelle erstellen
Kreuztab = pd.crosstab(ChristmasSales['Weather'], ChristmasSales['Category'])

# Pearson Koeffizient berechnen
pearson = contingency.association(KreuztabAufgabeC, method='pearson')

# Korrekturfaktor berechnen (nExpr ist die kleinere Anzahl von Zeilen oder Spalten)
nExpr = min(Kreuztab.shape)
c_korr = pearson * np.sqrt(nExpr / (nExpr - 1))

print(f'Pearson korrigiert: {c_korr:.4f}')
# c_korr liegt zwischen 0 und 1.
#0: Kein Zusammenhang zwischen beider Merkmale
#0,1 - 0,3: schwacher Zusammenhang
#0,3 - 0,5: Mittlerer Zusammenhang
#>0,5: Starker Zusammenhang
#1: Perfekter Zusammenhang

# BINOMIALVERTEILUNG (Ziehen mit Zurücklegen)
# n=Versuche, p=Wahrscheinlichkeit, k=Treffer
p_genau_k = binom.pmf(k, n, p) # Genau k Treffer
p_max_k = binom.cdf(k, n, p) # Höchstens k Treffer
p_min_k = 1 - binom.cdf(k-1, n, p) # Mindestens k Treffer (Gegeneignis)

# NORMALVERTEILUNG (norm)
# loc = Mittelwert (mu), scale = Standardabweichung (sigma)
prob = norm.cdf(x, loc=110, scale=5) # Wahrscheinlichkeit für Wert <= x
wert = norm.ppf(0.95, loc=110, scale=5) # Welcher Wert schneidet die unteren 95% ab?

# T-VERTEILUNG (wichtig bei kleinen Stichproben n < 30)
# df = Freiheitsgrade (n-1)
t_wert = t.ppf(0.975, df=29) # Kritischer t-Wert für zweiseitiges 95% KI

# HYPERGEOMETRISCHE VERTEILUNG (Ziehen ohne Zurücklegen, z.B. Qualitätskontrolle)
# M: Gesamtanzahl Objekte, n: Anzahl Objekte mit Eigenschaft, N: gezogene Stichprobe
p_hyper = hypergeom.pmf(k, M, n, N)

# CHI-QUADRAT TEST (Unabhängigkeitstest)
# Prüft, ob zwei kategoriale Variablen (z.B. Weather/Category) unabhängig sind
chi2_stat, p_val, dof, expected = contingency.chi2_contingency(Kreuztab)
print(f"p-Wert des Tests: {p_val:.4f}")
# p < 0.05 -> Zusammenhang ist signifikant (nicht zufällig)

# 1. Sollwert-Test bei BEKANNTEM Sigma (Z-Test)
# Man nutzt norm, da die Standardabweichung der Grundgesamtheit bekannt ist.
z_stat = (df['Spalte'].mean() - sollwert) / (sigma / np.sqrt(len(df)))
p_val = 2 * (1 - norm.cdf(abs(z_stat))) # Zweiseitig

# 2. Sollwert-Test bei UNBEKANNTEM Sigma (t-Test)
# Standardfall in den Übungen. Sigma wird durch die Stichprobe geschätzt.
# Nutzt pg.ttest (liefert p-val und Konfidenzintervall automatisch)
test = pg.ttest(df['length'], 115) # 115 ist der Sollwert

# KonfInter für Mittelwert (mu) bei geschätztem Sigma
# Nutzt die t-Verteilung. confint liefert Unter- und Obergrenze.
ki_mu = pg.ttest(df['length'], 0).confint

# KonfInter für Standardabweichung (sigma) - 95%
# Basiert auf der Chi-Quadrat-Verteilung.
n = len(df)
s2 = df['length'].var()
df_deg = n - 1
# Grenzen berechnen
chi2_low, chi2_high = chi2.interval(0.95, df_deg, df_deg)
sigma_lower = np.sqrt(df_deg * s2 / chi2_high)
sigma_upper = np.sqrt(df_deg * s2 / chi2_low)
print(f"95% KI für Sigma: [{sigma_lower:.4f}, {sigma_upper:.4f}]")

Z-Test: $mu$ prüfen, wenn $sigma$ der gesamten Fabrik/Population bekannt ist.
t-Test: $mu$ prüfen, wenn du $sigma$ erst aus deinen Daten berechnen musst.
KI Sigma: Bereich, in dem die echte Streuung der Produktion liegt.
Stichprobenumfang: Verhindert, dass du zu wenige Daten für eine sichere Aussage hast.
Je kleiner der Fehler $epsilon$ sein soll, desto größer muss $n$ sein.
```

```
THEORIE MITTELWERTE:
Arithmetisches Mittel: Der klassische Durchschnitt. Alle Werte werden addiert und durch die Anzahl geteilt. Du nutzt diese Berechnung bei normalen metrischen Daten ohne extreme Ausreißer. Beispiel: Das durchschnittliche Gewicht von Pinguinen.

Harmonisches Mittel: Der Durchschnitt für Raten und Verhältnisse. Du nutzt diese Berechnung bei Geschwindigkeiten oder Leistungsdaten bei einer konstanten Bezugsgröße. Beispiel: Du fährst 100 Kilometer mit 50 km/h und die gleiche Strecke zurück mit 100 km/h. Das harmonische Mittel liefert die korrekte Durchschnittsgeschwindigkeit.

Geometrisches Mittel: Der Durchschnitt für prozentuales Wachstum. Du nutzt diese Berechnung für Zinsseszinsen oder Wachstumsraten über mehrere Jahre.

import scipy.stats as stats
mittel_arith = df['Spalte'].mean()

mittel_harm = stats.hmean(df['Spalte'].dropna()) #keine Nullwerte und negative Zahlen!
mittel_geom = stats.gmean(df['Spalte'].dropna()) #Daten müssen positiv sein.
```

```
AGEN DER WAHRSCHEINLICHKEIT
isexperiment hat mögliche Ergebnisse. Die Menge aller Ergebnisse ist der raum. Ein Ereignis ist eine exakte Teilmenge davon.

Iitionssatz. Er berechnet die Wahrscheinlichkeit für Ereignis A oder Ereignis B. Du lierst beide Wahrscheinlichkeiten und zieht die Schnittmenge ab. Treten A und B nie chzeitig ein, fällt die Schnittmenge weg.

lingte Wahrscheinlichkeit. Sie beschreibt die Wahrscheinlichkeit von Ereignis A unter fixen Bedingung von Ereignis B. Vorwissen verändert die Chance.

3. Unabhängigkeit. Zwei Ereignisse sind unabhängig. Das Eintreten des einen beeinflusst das andere nicht. Die Gesamtwahrscheinlichkeit ist das Produkt der Einzelwahrscheinlichkeiten.

4. Totale Wahrscheinlichkeit. Sie berechnet die Gesamtwahrscheinlichkeit eines Ereignisses über verschiedene mögliche Pfade.

5. Satz von Bayes. Diese Formel kehrt bedingte Wahrscheinlichkeiten um. Du kennst die Wahrscheinlichkeit für ein Symptom bei einer Krankheit. Du berechnest damit die Wahrscheinlichkeit für die Krankheit bei genau diesem Symptom.
```

```
ZUFALLSVARIABLEN UND GRENZWERTSATZE
Eine Zufallsvariable wandelt ein Ereignis in eine Zahl um.

1. Erwartungswert. Dies ist der theoretische Durchschnittswert bei unendlich vielen Wiederholungen. Bei einem fairen Würfel liegt dieser Wert bei 3.5.

2. Gesetz der großen Zahlen. Du wiederholst ein Experiment oft. Der berechnete Durchschnitt nähert sich dem echten Erwartungswert immer genauer an.

3. Zentraler Grenzwertsatz. Du addierst viele unabhängige Zufallsvariablen oder bildest den Durchschnitt. Das Ergebnis nähert sich einer Normalverteilung an. Das gilt vollkommen unabhängig von der ursprünglichen Verteilung der Daten. Du benötigst eine ausreichend große Stichprobe. Die Regel verlangt mindestens 30 Datenpunkte.
```

```
DESKRIPTIVE STATISTIK (LAGE & STREUUNG)
• Mittelwert (Mean): Der Durchschnitt. Er ist extrem anfällig für Ausreißer.
• Median: Die exakte Mitte der Daten (50% sind kleiner, 50% sind größer). Er ist robust gegen Ausreißer. Nutze ihn bei schiefen Verteilungen (z. B. Einkommen).
• Standardabweichung (Std): Die durchschnittliche Abweichung aller Messwerte vom Mittelwert. Sie hat dieselbe Einheit wie die Daten (z. B. Meter).
• Varianz (Var): Die quadrierte Standardabweichung. Sie ist schwer praktisch zu interpretieren, wird aber für mathematische Formeln benötigt.
• Quantile / Quartile (Q1, Q3): Q1 schneidet die unteren 25% der Daten ab. Q3 schneidet die unteren 75% ab.
• Interquartilsabstand (IQR): Die Differenz aus Q3 und Q1 (Q3 - Q1). Er beschreibt die Streuung der mittleren 50% deiner Daten und bildet die Box im Boxplot. Er ist ein robustes Streuungsmaß.
```

```
HYPOTHESENTESTS (PINGUIN OUTPUT)
# T / z (Teststatistik): Ein mathematischer Zwischenwert, der aus den Daten berechnet wird, um den p-Wert zu ermitteln.
# dof (Freiheitsgrade): Hängt meist direkt mit der Stichprobengröße zusammen (oft n - 1).
# cohen-d (Effektstärke): Der p-Wert sagt nur, OB es einen Unterschied gibt. Cohen-d sagt, WIE GROSS dieser Unterschied in der Praxis ist.
• 0.2 = kleiner Effekt
• 0.5 = mittlerer Effekt
• 0.8 = großer Effekt
• power (Teststärke / 1 - Beta): Die Wahrscheinlichkeit, einen echten Effekt in den Daten auch tatsächlich zu finden. Ein Wert von über 0.8 (80%) gilt als gut.
```

```
KORRELATION VS. KAUSALITÄT (WICHTIGE FÄLLE)
• Korrelation: Zwei Dinge passieren gleichzeitig (z. B. Eisverkauf steigt und Sonnenbrand-Fälle steigen).
• Kausalität: Ein Ding verursacht das andere (Ursache und Wirkung).
• Regel für die Prüfung: Eine hohe Korrelation oder ein signifikantes Regressionsmodell beweist mathematisch niemals eine Kausalität. Es zeigt nur einen rechnerischen Zusammenhang.
```

```
#Bedingte Häufigkeit
# 1. Bedingung: Kategorie ist 'Food' -> Wie hoch ist der Anteil 'WAHR' (eingepackt)?
# Wir fixieren die Kategorie 'Food' (Zeile) und schauen auf das Geschenkpapier.
print(pd.crosstab(ChristmasSales['Category'], ChristmasSales['GiftWrap'], normalize='index').loc['Food', 'WAHR'])

# 2. Bedingung: Artikel ist 'FALSCH' (unverpackt) -> Wie hoch ist der Anteil 'Decorations'?
# Wir fixieren den Status 'unverpackt' (Zeile) und schauen auf die Kategorie.
print(pd.crosstab(ChristmasSales['GiftWrap'], ChristmasSales['Category'], normalize='index').loc['FALSCH', 'Decorations'])

# 3. Bedingung: Artikel ist 'WAHR' (eingepackt) -> Wie hoch ist der Anteil 'NICHT Toys'?
# Wir berechnen 1 minus den Anteil von Toys innerhalb der eingepackten Artikel.
print(1 - (pd.crosstab(ChristmasSales['GiftWrap'], ChristmasSales['Category'], normalize='index').loc['WAHR', 'Toys']))
#normalize='index': Berechnet die Häufigkeit relativ zur Zeile (die Bedingung steht vorne in der Klammer).
#.loc[Zeile, Spalte]: Greift gezielt auf den berechneten Prozentwert zu.
```

```
#Konfidenzintervall/Sigma schätzen + KI / KI für Sigma
# Daten
daten = np.array([4.90, 5.13, 4.95, 4.84, 5.00, 5.00, 4.93, 5.00, 4.93, 4.77])
n = len(daten)
x_bar = np.mean(daten)
alpha = 0.05

# Teil 1: o bekannt (alpha = 0.1 mg)
sigma = 0.1
z = norm.ppf(1 - alpha/2)
margin = z * sigma / np.sqrt(n)

lower = x_bar - margin
upper = x_bar + margin

print(f"Teil 1 - o bekannt:")
print(f"95%-KI: [{lower:.4f}, {upper:.4f}] mg")
print(f"~ Normalverteilung (z = {z:.3f})\n")

# Teil 2: o unbekannt (geschätzt)
s = np.std(daten, ddof=1)
t_wert = t.ppf(1 - alpha/2, df=n-1)
margin = t_wert * s / np.sqrt(n)

lower = x_bar - margin
upper = x_bar + margin

print(f"Teil 2 - o unbekannt (s = {s:.4f} mg):")
print(f"95%-KI: [{lower:.4f}, {upper:.4f}] mg")
print(f"~ t-Verteilung (t = {t_wert:.3f}, df = {n-1})\n")

# Teil 3: KI für o
chi2_lower = chi2.ppf(alpha/2, df=n-1)
chi2_upper = chi2.ppf(1 - alpha/2, df=n-1)

sigma_lower = np.sqrt((n-1) * s**2 / chi2_upper)
sigma_upper = np.sqrt((n-1) * s**2 / chi2_lower)

print(f"Teil 3 - KI für o:")
print(f"95%-KI: [{sigma_lower:.4f}, {sigma_upper:.4f}] mg")
print(f"~ x^2-Verteilung (df = {n-1})")
```

```
Schiefe = Adelie['Flipper Length (mm)'].skew()
Wölbung = Adelie['Flipper Length (mm)'].kurt()
print(Schiefe) # -0 » Nahezug symmetrisch
print(Wölbung) # -0 » normalähnliche Form
```

```
3 ∈ {1, 2, 3, 4} → „ist Element von“
{1, 2} ⊂ {1, 2, 3} → „Teilmenge, nicht gleich“
{1, 2} ∪ {2, 3} = {1, 2, 3} „Vereinigung“
{1, 2} ∩ {2, 3} = {2} „Schnittmenge“
{1, 2, 3} \ {2, 3} = {1} „Aus A, die nicht in B“
P(A|B) -> Wahrscheinlichkeit, dass A eintritt, unter der Bedingung, dass B bereits eingetreten ist.
```

Satz von Bayes

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

```
ChristmasSales = pd.read_csv('Christmas Sales and Trends.csv', sep=';', decimal=',')
display(ChristmasSales.head(3))
display(ChristmasSales.tail(3))

#für metrische Merkmale erkennen
ChristmasSales.describe()

#Häufigkeitsdiagramm TotalPrice
ChristmasSalesTotalPrice = ChristmasSales['TotalPrice']
display(ChristmasSalesTotalPrice.head(3))

sn.ecdfplot(data=ChristmasSalesTotalPrice)
```

```
plt.title('Kumuliertes Häufigkeitsdiagramm: TotalPrice')
plt.xlabel('Gesamtpreis')
plt.ylabel('Kumulierte Häufigkeit')
plt.grid(True)
plt.show()
print(f"Der Mindestwert der oberen 10% ist: {ChristmasSalesTotalPrice.quantile(0.9)}")

#Boxplot basierend auf Discount Amount und Event
plt.figure(figsize=(24, 6))
sn.boxplot(data=ChristmasSales, x='DiscountAmount', y='Event')
plt.show()

#Boxplot basierend auf Discount Amount und Event
plt.figure(figsize=(24, 6))
sn.boxplot(data=ChristmasSales, x='DiscountAmount', y='Event')
plt.show()
```

```
print("Kreuztabelle Giftwrap und Category")
KreuztabAufgabeA = pd.crosstab(ChristmasSales['GiftWrap'], ChristmasSales['Category'], margins = True)
print(KreuztabAufgabeA)

print("bedingte Häufigkeiten")
```

```
print("Food wird eingepackt:")
print(pd.crosstab(ChristmasSales['Category'], ChristmasSales['GiftWrap'], normalize='index').loc['Food', 'WAHR'])
print("unverpackt und aus Kategorie Decoration:")
print(pd.crosstab(ChristmasSales['GiftWrap'], ChristmasSales['Category'], normalize='index').loc['FALSCH', 'Decorations'])
print("eingepackt und nicht Kategorie Toys:")
print(1-(pd.crosstab(ChristmasSales['GiftWrap'], ChristmasSales['Category'],normalize = 'index' ).loc['WAHR', 'Toys']))

print("Kontingenzkoeffizient bestimmen und Ableiten")
KreuztabAufgabeC = pd.crosstab(ChristmasSales['Weather'], ChristmasSales['Category'], margins = True)
pearson = contingency.association(KreuztabAufgabeC,method='pearson')
nExpr = min(KreuztabAufgabeC.shape)
c_korr = pearson * np.sqrt(nExpr / (nExpr - 1))
print(f'Pearson korrigiert: {c_korr:.4f}')
#je kleiner Wert desto weniger zusammenhang
```

```
Ergebnis = []
for w1 in range(1,7):
    for w2 in range(1,7):
        Ergebnis.append(w1+w2)
Augensummen = pd.Series(Ergebnis).value_counts().sort_index()
```

```
x = Augensummen.index.values
absH = Augensummen.values
relH = absH/36

dataWürfel = pd.DataFrame(data={
    "Summe Augenzahl":x,
    "Abs. Häufigkeit":absH,
    "Rel. Häufigkeit":relH
})

print(dataWürfel)
```

```
# Es muss immer der Einsatz Abgezogen werden:
# Gewinn bei Augensumme 2: 10 - 3 = 7
# Gewinn bei Augensumme 11: 15 - 3 = 12
# Gewinn bei Augensumme 12: 15 - 3 = 12
# Gewinn restliche Augensummen: 0 - 3 = -3
Erwartungswert = (Gewinn mit rel. Häufig multipliziert je Augensumme und addiert)
print(f"Der Erwartungswert ergibt: {Erwartungswert:.4f}€")
```

```
print("Boxplot Länge nach Gruppe")
Rotor = pd.read_csv('Rotorblätter.csv')
display(Rotor.head(3))

# y ist die Länge, x die Gruppierung, hue sorgt für verschiedene Farben
sn.boxplot(data=Rotor, x='group', y='length', hue='group')
plt.show()
```

```
print("Konfidenzintervalle")
# Berechne Mittelwert, Standardfehler und Konfidenzintervalle pro Gruppe von Variable length
ci_tabelle = Rotor.groupby('group')['length'].apply(lambda x: pg.ttest(x, 0)).reset_index()
print(ci_tabelle[['group', 'estimate', 'confint']])
print("Das Intervall gibt den Bereich an, in dem der wahre Mittelwert der "
      "Grundgesamtheit mit 95% Wahrscheinlichkeit liegt. "
      "Überlappen sich die Intervalle der Gruppen nicht, ist der Längenunterschied "
      "zwischen ihnen statistisch signifikant.")
```

```
print("Hypothesentest ob Produktion neu kalibriert werden muss")

# Test gegen den festen Sollwert von 115 mit signifikanzniveau alpha = 0.05
test_ergebnis = pg.ttest(Rotor['length'], 115)
p_val = test_ergebnis['p-val'].values[0]
print(f"p-Wert: {p_val:.4f}")
if p_val < 0.05:
    print("Die Produktion weicht signifikant von 115m ab. Neukalibrierung nötig.")
else:
    print("Keine signifikante Abweichung festgestellt. Keine Neukalibrierung nötig.")
```

```
College = pd.read_csv('College.csv')
display(College.head(3))

print("\nn a")
# Berechnet Korrelationen zu 'Apps', sortiert sie absteigend und nimmt die Top 3 (ohne 'Apps' selbst)
top_3 = College.corr(numeric_only=True)['Apps'].sort_values(ascending=False).iloc[1:4]
print(top_3)
```

```
print("b")
# Regressionsmodell
# Daten vorbereiten (nur numerische Daten nutzen)
df_num = College.select_dtypes(include=[np.number])
x = df_num.drop('Apps', axis=1)
y = df_num['Apps']
# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Modell mit Konstante erstellen
X_train_const = sm.add_constant(X_train)
model = sm.OLS(y_train, X_train_const).fit()

print(model.summary())
Interpretation der Qualität: P-Werte (P>|t|$: Merkmale mit einem Wert unter 0.05 sind statistisch signifikant.R-squared: Gibt an, wie viel Prozent der Streuung der Bewerbungen durch das Modell erklärt werden.F-Statistik: Prüft, ob das Modell als Ganzes besser ist als ein Modell ohne Variablen.
```

```
c) Interpretation von R^2[.adj] = 0.91$
Ein Wert von 0.91 bedeutet, dass 91 % der Varianz der Zielvariablen (Apps) durch die unabhängigen Merkmale im Modell erklärt werden. Das Modell weist eine sehr hohe Güte auf. Die Vorhersagen liegen nah an den tatsächlichen Werten. Das "adj" (adjustiert) bedeutet, dass dieser Wert für die Anzahl der verwendeten Merkmale korrigiert wurde. Er ist somit aussagekräftiger als das normale R^2$, da er "Bestrafungen" für unnötige Variablen enthält.
```


Grenzwertsatz:
→ Egal wie die Daten in der Grundgesamtheit verteilt sind – wenn du viele Stichproben ziehst und jeweils den Mittelwert berechnest, dann sind diese Mittelwerte normalverteilt (Glockenform).

1.: Merkmale:
Qualitativ -> nicht messbar, Quantitativ -> messbar

Typ	messbar	Frage	Beispiel
Nominal	nein	Was für eine Art?	Haarfarbe, Blutgruppe, Geschlecht
Ordinal	nein	In welcher Reihenfolge?	Schulnoten, Platzierung, Likert-Skala
Diskret	ja	Wie viele? (nur ganze Zahlen)	Anzahl Kinder, Würfelaußen
Stetig	ja	Wie viele? (beliebige Werte)	Größe, Gewicht, Temperatur

Metrische Merkmale sind Diskret + Stetig zusammen
→ alle Merkmale, mit denen man wirklich rechnen kann! (Körpergröße, Gewicht, Einkommen, ...)

```
Imports
# === Grundlegende Datenverarbeitung ===
import pandas as pd
import numpy as np

# === Visualisierung ===
import seaborn as sea
import matplotlib.pyplot as plt
from matplotlib import cm, colors

# === Statistische Verteilungen (scipy.stats) ===
from scipy.stats import norm          # Normalverteilung
from scipy.stats import binom         # Binomialverteilung
from scipy.stats import hypergeom     # Hypergeometrische Verteilung
from scipy.stats import t             # t-Verteilung
from scipy.stats import chi2          # Chi²-Verteilung
from scipy.stats import gmean         # Geometrischer Mittelwert
from scipy.stats import contingency   # Kontingenzkoeffizient (Kreuztabelle)

# === Regression ===
import statsmodels.api as sm          # OLS-Regression
from sklearn.model_selection import train_test_split # Train/Test Split
from sklearn.linear_model import LinearRegression    # Lineare Regression (sklearn)

# === Sonstiges ===
import os                             # Dateisystem (Pfade prüfen)
import warnings
warnings.filterwarnings("ignore")
```

2.: Daten einlesen

```
Datensatz einlesen
data = pd.read_excel('./daten/all-weeks-countries.xlsx',
                    sheet_name='Sheetname') #falls nötig
data = pd.read_csv('./daten/all-weeks-countries.csv')

display(data.head(3)) # Ansicht erste Zeilen
display(data.tail(5)) # Ansicht letzte Zeilen
print('Größe des Datensatzes: {}'.format(netflix.shape))
display(data.info()) # Allgemeine Infos zum Datensatz
display(data.describe()) # statistische Kennzahlen zum Datensatz
```

```
Datensatz einlesen macht probleme?
import os
print(os.getcwd()) # Wo bist du gerade?
print(os.listdir('.')) # Existiert der Ordner "daten"?
print(os.listdir('./daten')) # Was ist in "daten" drin?
```

```
Allgemein
numerisch = data.select_dtypes(include=[np.number]).columns.tolist()
print("Numerische Merkmale:", numerisch)

data_column = data['TotalPrice']
print("oberen 10% der Ausprägungen", data_column.quantile(0.9))

median = data.groupby('Species')['Culmen Length (mm)'].median()
maximum = data.groupby('Species')['Culmen Length (mm)'].max()
minimum = data.groupby('Species')['Culmen Length (mm)'].min()
quantil025 = data.groupby('Species')['Culmen Length (mm)'].quantile(0.25)
quantil075 = data.groupby('Species')['Culmen Length (mm)'].quantile(0.75)
avg = data.groupby('Species')['Culmen Length (mm)'].mean()
var = data.groupby('Species')['Culmen Length (mm)'].var() #Was ist Varianz überhaupt? Ein Maß dafür, wie stark die Werte um den Mittelwert streuen.
```

Lage und Streumaß:

```
Standardisierte Werte
# Standardisierung der Daten am Beispiel der Culmen Depth

feature = 'Culmen Depth (mm)'
for pingu, data in datensatz.groupby('Species'):
    # daten in ein numpy-array abspeichern
    orig = np.array(data[feature].values)
    mean = orig.mean()
    # Standardabweichung
    std = orig.std(ddof=1)
    # transformierte Daten
    transf = (orig - mean)/std
    fig, axs = plt.subplots(1,2,figsize=(12,4))
    sea.scatterplot(x=range(orig.shape[0]), y=orig, ax=axs[0])
    sea.scatterplot(x=range(transf.shape[0]), y=transf, ax=axs[1])
    axs[0].set_title('mean = {0:.2f}, std = {1:.2f}'.format(mean, std))
    axs[1].set_title('mean = {0:.2f}, std = {1:.2f}'.format(transf.mean(), transf.std(ddof=1)))
    axs[0].grid()
    axs[1].grid()
    fig.suptitle(pingu + ' ' + feature)
    plt.show()
```

```
geometrischer und harmonischer Mittelwert -> mit
# Geometrischer Mittelwert für prozentuales Wachstum
# sequenz von Zufallszahlen als Wachstumsfaktoren
rng = np.random.default_rng(seed=2024) # seed erzeugt wiederholbares Ergebnis
changes = rng.normal(loc=1, scale=0.1, size=100) # 100 Zufallszahlen basierend auf einer Normalverteilung
# loc=1 -> Mittelwert=1 (im Schnitt kein Wachstum/Verlust)
# scale=0.1 -> Standardabweichung=0.1 -> Schwankung +/- 10%
# alle Werte sind etwa mit +/- 10% -> 1.05, 0.95, 1.12, 0.98

werte = [100] # Anfangswert ist 100
werte.extend(100*np.cumprod(changes)) # [100, 100*w1, 100*w1*w2, 100*w1*w2*w3 ...]

# Arithmetrischer Mittelwertes der Wachstumsfaktoren (Durchschnitt aller 100 Werte)
changes_mean = np.mean(changes)

# Geometrischer Mittelwertes
changes_gmean = gmean(changes)
print('Wachstumsrate (arithmetrisch): {0:.4f}'.format(changes_mean))
print('Wachstumsrate (geometrisch): {0:.4f}'.format(changes_gmean))

fig, ax = plt.subplots(1,1,figsize=(10,4))
ax.plot(range(101), werte, label='originaler Verlauf')
ax.plot(range(101), [100*changes_mean**i for i in range(101)], label='Verlauf mit arithmetrischen Mittelwert')
ax.plot(range(101), [100*changes_gmean**i for i in range(101)], label='Verlauf mit geometrischen Mittelwert')
ax.grid()
ax.set_ylabel('Wert')
ax.set_xlabel('zeitlicher Verlauf')
ax.legend()
plt.show()

# Arithmetisch: Für Durchschnittsnoten
# Geometrisch: Für Wachstumsraten, Renditen/ Zinsen, Aktienkurse
```

Modus Was: Häufigster Wert im Datensatz.
Median = 5 Was: Mittlerer Wert der sortierten Daten.
IQR = 3 Was: Interquartilsabstand – Streuung der mittleren 50% der Daten. Berechnen: IQR = $\bar{y}_{0.75} - \bar{y}_{0.25}$
Interpretieren: Die mittleren 50% der Werte liegen in einem Bereich von 3 Einheiten. Robust gegen Ausreißer – besser als Spannweite.
Arithmetisches Mittel $\bar{y} = 5$: Berechnen: $\bar{y} = (x_1 + x_2 + \dots + x_n) / n$ Interpretieren: Langfristiger Durchschnitt. Nicht robust gegen Ausreißer. Nur für metrische Daten sinnvoll.
Spannweite $R = 10$ Was: Abstand zwischen kleinstem und größtem Wert. Berechnen: $R = y_{(n)} - y_{(1)}$
Interpretieren: Gesamter Wertebereich der Daten beträgt 10. Sehr sensitiv für Ausreißer – ein extremer Wert verändert R komplett.
Varianz $s^2 = 4$ Was: Mittlere quadratische Abweichung vom Mittelwert. Berechnen: $s^2 = \sum (y_i - \bar{y})^2 / (n-1)$
Interpretieren: Werte weichen im Quadrat durchschnittlich um 4 ab. Einheit ist quadratisch –> schwer direkt lesbar, daher Standardabweichung bevorzugen.
Standardabweichung $s = 2$ Was: Wurzel der Varianz – typische Abweichung vom Mittelwert in der Originaleneinheit. Berechnen: $s = \sqrt{s^2}$ Interpretieren: Werte weichen typischerweise ±2 vom Mittelwert ab. Bei Normalverteilung: ~68% in $[\bar{y}-s, \bar{y}+s]$, ~95% in $[\bar{y}-2s, \bar{y}+2s]$.
Gini-Koeffizient $G = 0,6$ Was: Maß für Ungleichverteilung einer Merkmalsausprägung. Berechnen: $G^* = (n+1-2T)/(n-1)$, grafisch als Fläche zwischen Lorenzkurve und Diagonale. Interpretieren: $G^* = 0$ → alle haben gleich viel. $G^* = 1$ → eine Einheit hat alles. 0,6 = starke Konzentration.
Chi2 $\chi^2 = 0$ Was: Abweichung der beobachteten Häufigkeiten von den bei Unabhängigkeit erwarteten. Berechnen: $\chi^2 = \sum (h_{ij} - \hat{h}_{ij})^2 / \hat{h}_{ij}$ mit $\hat{h}_{ij} = h_{i+} \cdot h_{+j} / n$ Interpretieren: $\chi^2 = 0$ → vollständige Unabhängigkeit. Größer = stärkerer Zusammenhang – aber Wert hängt von n ab, daher nicht direkt zwischen Datensätzen vergleichbar.

3.:Häufigkeiten:

Begriff	Frage	Beispiel	Spaltenbeispiele
Absolute	Wie viele?	8 Filme	5, 8, 4, 3
Relative	Wie viel %?	40%	25%, 40%, 20%, 15%
Kommulierte	Wie viel % bis hierhin?	65%	25%, 65%, 85%, 100%

```
Häufigkeiten Absolut/ Relativ/ Kommulativ
# 1.Datensatz reduzieren durch Filter und 2.Tabellarisch anzeigen
data_lastweek = data[data['week']=='2024-08-18']
data_reduced = data_lastweek[(data_lastweek['category']=='Films') & (data_lastweek['weekly_rank']==1)]
data_reduced.value_counts('show_title') #2.zeigt die Anzahl von jedem Titel an

# — Plot 1: Absolute Häufigkeit
fig1, ax1 = plt.subplots(figsize=(12, 5))
sea.countplot(data=data_reduced, y='show_title', ax=ax1, color='deeppink')
ax1.set_title('Absolute Häufigkeit')
plt.tight_layout()
plt.show()

# — Plot 2: Relative
Häufigkeit
fig2, ax2 = plt.subplots(figsize=(12, 5))
sea.countplot(data=data_reduced, y='show_title', stat='percent', ax=ax2)
ax2.set_title('Relative Häufigkeit')
plt.tight_layout()
plt.show()

# — Plot 3: Kumulative Häufigkeit (eigene Figure)
data_rn = data[data['show_title']=='Red Notice']
fig3, ax3 = plt.subplots(figsize=(10, 5))
sea.ecdfplot(data=data_rn, x='weekly_rank', ax=ax3)
ax3.set_title('Kumulative Häufigkeit')
ax3.grid()
plt.tight_layout()
plt.show()
```

```
Scatterplot (Streudiagramm) -> zeigt die Beziehung zwischen zwei Merkmalen – jeder Punkt steht für eine Beobachtung.
fig3, ax3 = plt.subplots(figsize=(7, 7))
ax3.scatter(data['HutFarbe'], data['Lamellenfarbe'])
ax3.set_xlabel('HutFarbe')
ax3.set_ylabel('Lamellenfarbe')

plt.xticks(rotation=45) # ersetzt fig.autofmt_xdate()
plt.tight_layout() # verhindert abgeschnittene Beschriftungen
plt.show()
```

```
Liniengraphen
fig = plt.figure(figsize=(12,6))
ax = fig.add_subplot(111)

ax.plot(range(data.shape[0]), data['Culmen Length (mm)'], label='culmen length')
ax.plot(range(data.shape[0]), data['Culmen Depth (mm)'], label='culmen depth', marker='o', markevery=10, markersize=5)

ax.grid()
ax.set_xlabel('Pinguin Nr.')
ax.set_ylabel('Länge in mm')
ax.legend()
plt.show()
```

```
Tortendiagramm
colors = data['Sporenpulverfarbe'].value_counts() #gibt: weiß 100, braun 50, ...
colors = colors.sort_values(ascending=False) #-> weiß gibts z.B.: 100 mal

# 4 Kategorien und eine sonstige
colorsAgg = colors.iloc[:4]
colorsAgg.loc['Other'] = colors.iloc[4:].sum()

fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(111)
ax.pie(colorsAgg.values, normalize=True, labels=colorsAgg.index, startangle=90, autopct='%1.2f%%')
plt.show()
```

```
Regression:
korrelation von Spalten
korr = data.corr(numeric_only=True)['Apps'].drop('Apps').abs().sort_values(ascending=False)
print(korr.head(3)) # 3 Spalten welche am meisten korrelieren
```

```
Regression(Tabelle)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# — Daten vorbereiten
data_model = pd.get_dummies(data, columns=['Private'], drop_first=True)
data_model = data_model.astype(float) # * DAS ist der Fix! Alles zu float konvertieren

X = data_model.drop(columns=['Apps'])
y = data_model['Apps']
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8, random_state=42)

# — statsmodels Regression
X_train_sm = sm.add_constant(X_train, prepend=True)
X_test_sm = sm.add_constant(X_test, prepend=True)

model_sm = sm.OLS(y_train, X_train_sm).fit()
print(model_sm.summary())
```

R² wenn hoch dann gut -> 93% wäre z.B.: gut -> da das Modell dann 93% der Varianz erklärt
Ein Merkmal ist signifikant wenn $P>|t| < 0.05$
Wenn ein Merkmal $P>|t| > 0.05$ -> dann trägt dieses kaum zur Vorhersage bei!
Wenn R²_adj eine kleine Abweichung von R² hat: dann: Funktioniert das Modell auf neuen Daten fast genauso gut wie auf Trainingsdaten.
Wenn R²_adj = 0.90, dann sind nur 10% Varianz welche unerklärt bleiben!

```
Lineare Regression
var = sm.add_constant(x_train, prepend=True)
model = sm.OLS(y_train, var)
res = model.fit()
display(res.summary())
```

Kontingenzkoeffizient $C = 0 / \bar{C} = 1$ Was: Normierte Kennzahl für Zusammenhang zwischen zwei nominalen Merkmalen. Berechnen: $C = \sqrt{\chi^2 / (n + \chi^2)}$. Korrigiert: $\bar{C} \in [0,1]$. Interpretieren: $\bar{C} = 0$ → vollständig unabhängig. $\bar{C} = 1$ → vollständig abhängig. Vergleichbar zwischen verschiedenen Tabellen.
Kovarianz $s_{xy} = 3$ Was: Gemeinsame Streuung zweier metrischer Merkmale. Berechnen: $s_{xy} = (1/(n-1)) \cdot \sum (x_i - \bar{x})(y_i - \bar{y})$ Interpretieren: $s_{xy} > 0$ → steigen gemeinsam. $s_{xy} < 0$ → gegenläufig. $s_{xy} = 0$ → kein linearer Zusammenhang. Wert hängt von Einheiten ab → nicht zwischen Datensätzen vergleichbar.
Pearson-Korrelation $r_{xy} = 0,85$ Was: Normierte Kovarianz – Stärke und Richtung des linearen Zusammenhangs. Wertebereich $[-1, 1]$. Berechnen: $r_{xy} = s_{xy} / (s_x \cdot s_y)$ Interpretieren: 0,85 → starker positiver linearer Zusammenhang. -0,85 → stark negativ. 0 → kein linearer Zusammenhang (aber evtl. nichtlinearer!). $|r| \leq 0,3$ schwach, $|r| \geq 0,8$ stark. Korrelation ≠ Kausalität!
Spearman ρ_s Was: Korrelation auf Basis von Rängen – für ordinale Daten oder bei Ausreißern. Berechnen: Pearson-Formel auf die Ränge $R(x_i), R(y_i)$ anwenden. Interpretieren: Misst Monotonie statt Linearität. $r_{xy} = 0$ möglich obwohl $\rho \neq 0$ – Spearman erkennt auch nichtlineare monotone Zusammenhänge.
 $\beta_0 = 3$ Was: Steigung der Regressionsgeraden – Einfluss von x auf y. Berechnen: $\beta_1 = s_{xy} / s_x^2 = r \cdot (s_y/s_x)$. $\beta_0 = \bar{y} - \beta_1 \bar{x}$ Interpretieren: Steigt x um 1 Einheit, steigt y im Durchschnitt um 3. β_0 = Wert von y wenn x = 0. Gerade geht immer durch $(\bar{x}, \bar{y})$.
Residuum $e_i = y_i - \hat{y}_i = 2$ Was: Abweichung zwischen echtem Wert und Modellvorhersage. Berechnen: $e_i = y_i - \hat{y}_i$ Interpretieren: Modell liegt bei dieser Beobachtung um 2 Einheiten daneben. Ziel: Residuen klein, zufällig verteilt, kein Muster.
p-Wert eines Koeffizienten $< 0,05$ Was: Wahrscheinlichkeit, diesen Koeffizient zu sehen wenn er eigentlich 0 wäre. Berechnen: $t = \beta_j / SE(\beta_j)$, dann p-Wert aus t-Verteilung. Interpretieren: $p < 0,05$ → Variable hat signifikanten Einfluss auf y. $p > 0,05$ → Variable trägt nichts bei, Kandidat zum Entfernen.
F-Test $p < 0,05$ Was: Testet ob das gesamte Modell etwas erklärt. Berechnen: $F = ((n-K-1)/K) \cdot ESS/RSS$ Interpretieren: $p < 0,05$ → mindestens eine Variable hat Erklärungskraft. Wichtig bei multipler Regression – einzelne p-Werte allein reichen nicht.
RSE = 50 Was: Residual Standard Error – typische Abweichung der Vorhersage vom echten Wert. Berechnen: $RSE = \sqrt{RSS/(n-2)}$ Interpretieren: Vorhersage liegt typischerweise ±50 Einheiten daneben. In der Einheit von y – direkt interpretierbar.
 μ – Erwartungswert Was: Theoretischer Mittelwert der Grundgesamtheit. Berechnen: Diskret: $E(X) = \sum x_i \cdot P(X=x_i)$. Stetig: $E(X) = \int x \cdot f(x) dx$. Interpretieren: Der Wert, um den X langfristig im Durchschnitt schwankt. Unbekannt – wird durch $\bar{x}$ geschätzt.
Konfidenzintervall Was: Intervall, das den wahren Parameter mit Wahrscheinlichkeit $(1-\alpha)$ enthält. Berechnen:

- σ bekannt: $\bar{x} \pm 1,96 \cdot \sigma/\sqrt{n}$
- σ unbekannt: $\bar{x} \pm t_{0,95} \cdot s/\sqrt{n}$ (t statt z, weil mehr Unsicherheit → breiteres Intervall)
- für σ : $\sqrt{[(n-1)s^2/\chi^2_{\text{oben}}] - [(n-1)s^2/\chi^2_{\text{unten}]}$

Interpretieren: Bei 100 Wiederholungen würden ~95 der berechneten Intervalle den wahren Wert enthalten. Nicht: „ μ liegt mit 95% in diesem Intervall“ – μ ist fix, das Intervall ist zufällig!
 α – Signifikanzniveau Was: Tolerierte Wahrscheinlichkeit für einen Fehler 1. Art (H_0 fälschlich ablehnen). Berechnen: Wird vor dem Test festgelegt – typisch $\alpha = 0,05$. Interpretieren: $\alpha = 0,05$ bedeutet: man akzeptiert 5% Risiko, H_0 zu Unrecht abzulehnen. Kleineres α = konservativer, aber schwerer signifikant zu werden.

```
Box-Plot
fig, ax = plt.subplots(figsize=(10, 6))
palette = {"Black Friday": "#e74c3c", "Christmas Market": "#2ecc71"}
sea.boxplot(data=data, x="Event", hue="Event", y="DiscountAmount", palette=palette, ax=ax, legend=False)
# x=Event gibt an das in der Spalte Event "Black Friday" und "Christmas ..." verwendet wird
ax.set_title("Boxplot: DiscountAmount nach Event")
ax.set_xlabel("Event")
ax.set_ylabel("DiscountAmount (€)")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# Box liegt in den mittleren 50% wobei hier noch ein Strich drinnen ist(Median)
# Die Wisker(Striche außen) kennzeichnen das unter/ obere Normal
# -> wenn die Wisker lange sind gibt es eine große Streuung der Werte
# Die Kreise Kennzeichnen Außreißer
```

```
4.: Kreuztabelle
Kreuztabelle
ct = pd.crosstab(data['GiftWrap'], data['Category'], margins=True)
display(ct)

#bedingte Häufigkeiten -> wie oft eine Bedingung zutrifft
food_wrapped = ct.loc[True, "Food"] #Zeile, Spalte (Y, X)
food_total = ct.loc["All", "Food"]
p_wrap_food = food_wrapped / food_total
print(f"P(GiftWrap=True | Food) = {food_wrapped}/{food_total}"
      f" = {p_wrap_food:.4f} ({p_wrap_food*100:.2f} %)")

#korrigierter Kontingenzkoeffizient -> Zusammenhang zwischen Spalten
ct2 = pd.crosstab(data["Weather"], data["Category"])
print("Kontingenztafel Weather x Category:")
print(ct2)
n = data.shape[0] # speichert Anzahl Zeilen
pearson = contingency.association(ct2, method="pearson")
nExpr = min(ct2.shape) # min(Zeilen, Spalten)
pearson_k = pearson * np.sqrt(nExpr / (nExpr - 1))

print(f"\nPearson (unkorrigiert) : {pearson:.4f}")
print(f"Pearson (korrigiert) : {pearson_k:.4f} (max. 1)")
print("Interpretation: Wert nahe 0 -> kein nennenswerter Zusammenhang"
      " zwischen Wetter und Kategorie.")
```

```
Jointplot (Randverteilung mit Jointplot)
# Randverteilungen mit jointplot
# z.B.: Plot mit 2 Spalten und 10 Zeilen
# ganz oben zeigen zusätzlich 2 Balken die Verteilung in den Spalten an
# und Rechts gibt ein Balken pro Zeile welche die Verteilung der Zeilen anzeigt
fig = sea.jointplot(data=data, x='Klasse', y='Lamellenfarbe', kind='hist')
fig.ax_joint.set_xticklabels(fig.ax_joint.get_xticklabels(), rotation=45)
plt.show()
```

```
Pairplot -> als anschauliche Darstellung der Zusammenhänge
penguins = pd.read_excel('./daten/penguinsSmall.xlsx') #, index_col=0) erste Spalte als Index

# pairplot als anschauliche Darstellung der Zusammenhänge
sea.pairplot(data = data, kind='scatter', corner=True, hue='Species', height=1.75)
plt.show()
```

```
Violinplot
fig, ax = plt.subplots(figsize=(12,4), sharey=True)
sea.violinplot(data=data, y='Body Mass (g)', hue='Species', legend='auto', ax=ax, inner='stick', cut=0.)
# hue='Species' -> jede Pinguinart kriegt eine eigene Violine, inner='stick' -> zus. jeder Datenpunkt als Strich
# cut=0. -> jede Violine endet bei Min/Max

ax.grid()
fig.suptitle('Alle Pinguinarten')
plt.show()
```

Statistik:

Begriff	Anz. Variablen	Variablen	Beispiel genauer
Univariant	1	nur Preis	Wie ist der Preis verteilt?
Bivariant	2	Preis + Alter	Hängt Preis vom Alter ab?
Multivariant	3+	Preis + Alter + Region	zahlen Kunden aus d. Norden mehr?

```
Normalverteilung (mit Variation des Mittelwerts)
mu = 145
sigma = 2

x = np.linspace(mu-4*sigma, mu+4*sigma, 500) #geniert 500 Datenpunkte
pdf = norm.pdf(x, mu, sigma) # berechnet Wahrscheinlichkeitsdichtefunktion
cdf = norm.cdf(x, mu, sigma) # berechnet kumulative Verteilungsfunktion
plt.figure(figsize=(12,5))

plt.subplot(1,2,1) #Mehrfachplotten
plt.plot(x, pdf)
plt.title("PDF der Normalverteilung")
plt.xlabel("Volumen [ccm]")
plt.ylabel("pdf")
plt.grid(True)

plt.subplot(1,2,2) #Mehrfachplotten
plt.plot(x, cdf)
plt.title("CDF der Normalverteilung")
plt.xlabel("Volumen [ccm]")
plt.ylabel("cdf")
plt.grid(True)
plt.show()
```

```
Numerische Aussagen erfragen
# Wahrscheinlichkeit, eine zufällig gewähltes Objekt < 145 ist
dist = norm(loc=mu, scale=sigma)
p_b = round(dist.cdf(145), 4)
print(p_b)

# Wahrscheinlichkeit, eine zufällig gewähltes Objekt > 155 ist
p_c = round(1 - dist.cdf(155), 4)

# Wahrscheinlichkeit, eine zufällig gewähltes Objekt > 150 && < 155
p_d = round((dist.cdf(155) - dist.cdf(150)), 4)

#: Welches Maximalvolumen wird von 90% der Objekte unterschritten?
x_90 = round(dist.ppf(0.9), 4)

#: Welches Minimalvolumen wird von 90% der Objekte überschritten?
x_10 = round(dist.ppf(0.1), 4)
```

```
Variation des Mittelwerts
x = np.linspace(-1, 12, 500)
mus = [3, 4, 5, 10]
sigma = 0.5

plt.figure(figsize=(8,5))
for mu in mus:
    plt.plot(x, norm.pdf(x, mu, sigma), label=f"μ={mu}, σ={sigma}")
```

```
Chi Quadrat (X²-Verteilung)
nus = [2, 3, 5, 10, 20, 30] # Freiheitsgrade
x = np.linspace(0, 50, 1000) # x-Achse

plt.figure(figsize=(10, 4))
for nu in nus:
    dist = chi2(df=nu)
    plt.plot(x, dist.pdf(x), label=f'ν = {nu}') #pdf/ cdf
```

Operation Symbol Bedeutung
Durchschnitt $A \cap B$, „x gehört sowohl zu A als auch zu B“ – die Schnittmenge
Vereinigung $A \cup B$, „x gehört zu A oder zu B (oder beidem)“
Differenz $A \setminus B$, „x ist in A, aber nicht in B“
Komplement $\bar{A}$, „Alles in Ω , das nicht in A liegt“ – Gegenereignis
Teilmenge $A \subseteq B$, „Jedes Element von A ist auch in B“
Bedingte Wahrscheinlichkeit $P(B|A) = 0,7$ Was: Wahrscheinlichkeit von B, gegeben dass A bereits eingetreten ist. Berechnen: $P(B|A) = P(A \cap B) / P(A)$. Umgestellt: $P(A \cap B) = P(A) \cdot P(B|A)$. Interpretieren: Wenn A passiert ist, tritt B mit 70% Wahrscheinlichkeit auf. Basis für Wahrscheinlichkeitsbäume